



Infrastruktura i servisi u oblaku

Projekt br.2

Student: Edin Ahmetbegović

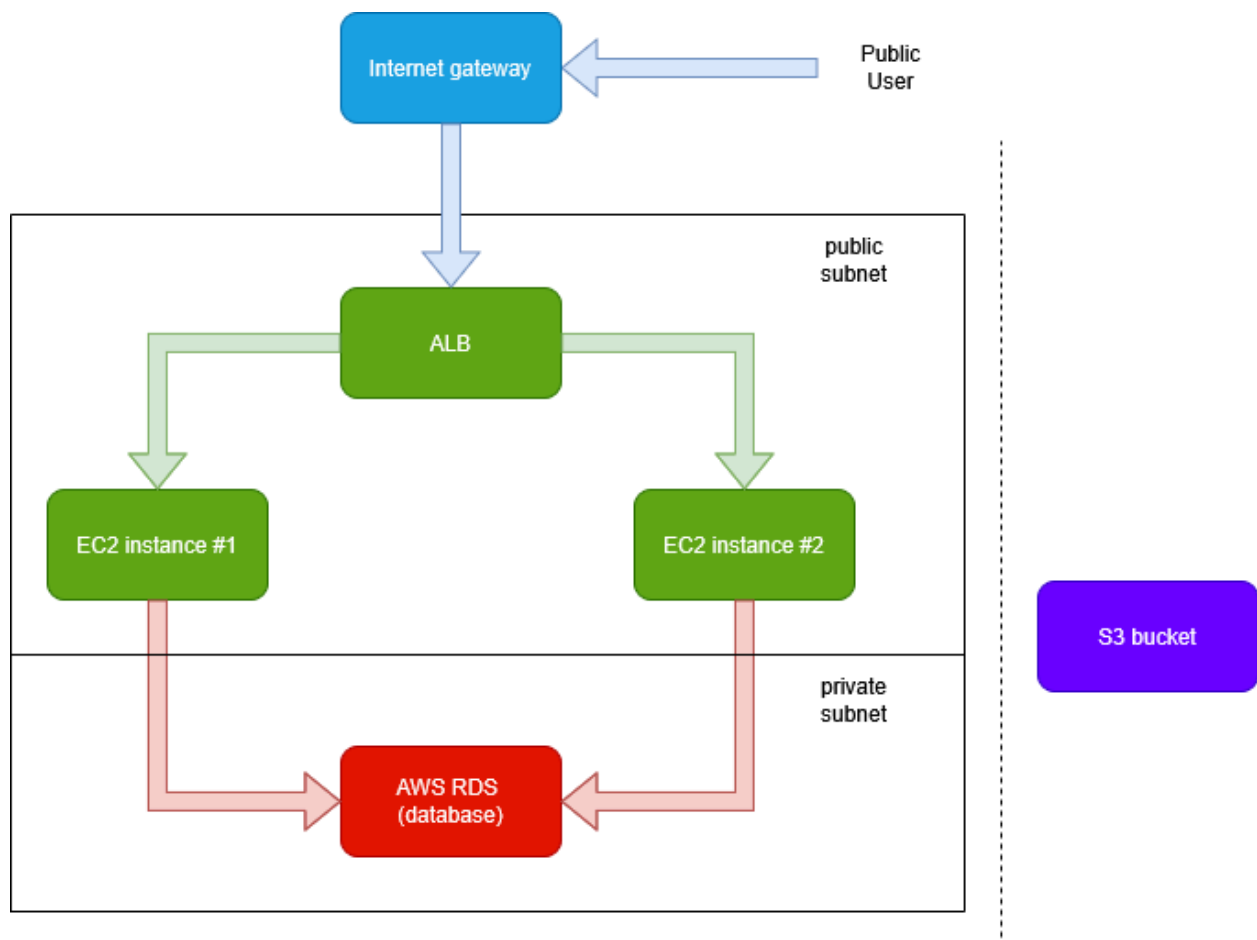
Tuzla, April 2026 god.

Uvod

Ovaj projekat je web aplikacija tematski bazirana na japanskom Sengoku periodu (1467–1603). Prikazuje interaktivnu mapu Japana sa dugmićima klanova koji pri kliku učitavaju detaljne informacije o svakom klanu iz PostgreSQL baze podataka. Aplikacija se sastoji od Flask backend-a, Nginx frontend-a i PostgreSQL baze podataka, sve pokrenuto na Amazon Web Services (AWS) platformi koristeći Docker kontejnere. Primarni fokus ovog projekta je demonstracija cloud infrastrukture koristeći AWS servise i Terraform Infrastructure as Code automatizaciju.

Arhitekturni Dijagram

Arhitektura prati dijamantni oblik sa jednom javnom tačkom pristupa. Javni korisnik pristupa aplikaciji kroz Application Load Balancer (ALB) koji raspoređuje saobraćaj između dvije EC2 instance radi visoke dostupnosti. Obje EC2 instance pokrenute su sa identičnim Docker kontejnerima (frontend nginx + backend Flask) i spajaju se na jednu zajedničku RDS PostgreSQL bazu podataka smještenu u privatnoj pod mreži. Statički resursi kao što su slike servuju se direktno iz S3 bucketa. Cijela infrastruktura se nalazi unutar prilagođenog VPC-a sa dvije javne pod mreže (po jedna za svaku EC2 instancu i ALB) i jednom privatnom pod mrežom (RDS).



Slika 2.1 Arhitektura AWS

Procjena Troškova AWS Servisa

Sljedeća je okvirna mjesečna procjena troškova za kontinuirano pokretanje ove aplikacije u us-east-1 regiji koristeći instance tipove prihvatljive za free tier.

- EC2 - dvije t2.micro instance - oko 8.50 USD mjesečno (0.0116 USD po satu svaka)
- RDS - jedna db.t3.micro PostgreSQL instanca - oko 12.50 USD mjesečno (0.017 USD po satu)
- Application Load Balancer - oko 16.20 USD mjesečno (0.008 USD po satu plus 0.008 USD po LCU)
- S3 - statične slike ukupne veličine ispod 50MB - ispod 0.01 USD mjesečno
- Data transfer - minimalan saobraćaj za studentski projekat - oko 1.00 USD mjesečno

Ukupna okvirna cijena: oko 38 USD mjesečno za kontinuirano pokretanje

Upute za Pokretanje i Testiranje

Softverski preduslovi

Za ručni deployment potrebno je imati instaliran Docker i Docker Compose na lokalnoj mašini, Git za kloniranje repozitorija, te pristup AWS Academy Sandbox okruženju. Za Terraform automatizaciju Terraform se instalira direktno u sandbox terminalu putem priložene skripte.

Pokretanje Terraform automatizacije

1. Pokrenuti AWS Academy Sandbox lab sesiju i sačekati da status postane "ready".
2. U sandbox terminalu otvoriti editor (Files tab), kreirati novi fajl pod nazivom setup.sh, te kopirati sadržaj fajla iz GitHub repozitorija (setup script/setup.sh) u njega i sačuvati.
3. U terminalu pokrenuti skriptu komandom: `bash setup.sh`

Skripta će automatski instalirati Terraform, provjeriti da li je Git instaliran, klonirati GitHub repozitorij i inicijalizirati Terraform.

4. Navigirati u Terraform folder komandom: `cd ISO_Sengoku_PR2/terraform-sengoku`
5. Pokrenuti infrastrukturu komandom: `terraform apply`

Upisati "yes" kada Terraform zatraži potvrdu. Proces traje oko 10-15 minuta, uglavnom zbog kreiranja RDS instance.

6. Nakon završetka Terraform će ispisati output vrijednosti uključujući ALB DNS adresu. Sačekati još 3-5 minuta da se Docker kontejneri pokrenu na EC2 instancama.

Pristup aplikaciji

Nakon što je deployment završen, aplikacija je dostupna putem ALB DNS adrese koja se ispisuje u Terraform outputu u formatu:

sengoku-alb-xxxxxxxxxx.us-east-1.elb.amazonaws.com

Ova adresa se otvara u bilo kojem web pretraživaču bez potrebe za HTTPS.

Izazovi i Rješenja

Tokom deployment-a naišlo se na nekoliko značajnih tehničkih problema.

S3 Bucket Terraform problem

Najveći izazov bio je kreiranje S3 bucketa putem Terraform koda. AWS Terraform provider automatski pokušava pročitati GetObjectLockConfiguration konfiguraciju svaki put kada kreira ili ažurira S3 bucket. AWS Academy Sandbox okruženje eksplicitno blokira ovu operaciju na nivou IAM politike, što uzrokuje grešku bez obzira na konfiguraciju Terraform koda. Problem je istraživao više od 10 sati uz isprobavanje različitih verzija providera, lifecycle blokova i alternativnih pristupa. Zaključak je da se radi o hardkodiranom ograničenju sandbox IAM politike koje se ne može zaobići. Rješenje je bilo kreiranje S3 bucketa direktno iz EC2 instance putem AWS CLI komandi unutar user_data skripte, budući da EC2 instance imaju LabInstanceProfile rolu koja posjeduje neophodne S3 permisije. Terraform kod za S3 bucket ostaje u main.tf fajlu ali je zakomentiran, što demonstrira ispravnu implementaciju koja bi radila na punom AWS accountu bez sandbox ograničenja.

Korištenje AI alata

Tokom izrade projekta korišten je AI alat (Claude, ChatGPT i Grok) kao pomoć pri učenju, razumijevanju AWS servisa, generisanju template koda i rješavanju tehničkih problema. AI je korišten kao interaktivni asistent - ne za direktno generisanje gotovih rješenja, već za vođeno učenje i razumijevanje svake komponente prije implementacije. Svaki korak je diskutovan i razjašnjen prije nego što je ikakav kod generisan ili primijenjen.

Primjeri korištenih promptova

- "Objasni mi šta je Terraform, kako radi i koja je sintaksa?"
- "Kako funkcioniše VPC sa javnim i privatnim subnetima u AWS-u?"
- "Zašto ALB treba dva subneta u različitim Availability Zones?"
- "Ako deployam aplikaciju na AWS, da li moram mijenjati adrese definirane u docker-compose.yml fajlu?"
- "Kako da napravim user_data skriptu koja automatski instalira Docker, klonira repo i pokrene docker-compose?"
- "Ako obje EC2 instance pokrenu istu skriptu u potpuno isto vrijeme zbog Terraforma, da li će if naredbe zaista funkcionisati i spriječiti duplikate?"
- "Zašto Terraform dobija grešku GetObjectLockConfiguration na S3 bucketu i kako to riješiti?"
- "Kako da napravim if/else u bash skripti da provjerom je li Git instaliran?"

Prihvaćeno i izmijenjeno

Generisani sadržaj korišten je kao osnova koja je zatim testirana, debugovana i prilagođena realnim ograničenjima sandbox okruženja.

Primjeri:

- Terraform fajlovi (main.tf, outputs.tf, variables.tf) su generisani od strane Claude AI kao osnovni templates koje sam naknadno modificirao i prilagodio specifičnim zahtjevima zadatka i ograničenjima sandbox okruženja.
- Za određene instalacije unutar user_data skripte nisam znao odakle i kako preuzeti potrebne alate, pa sam postavljao upite poput "kako da preuzmem najnoviju verziju Docker Compose za Linux" i koristio dobivene komande direktno u skripti.
- Claude AI je korišten kao alat za bugfixing i potvrdu ispravnosti koda, primjer toga je slanje cijelog main.tf fajla sa pitanjem "hoće li ova izmjena raditi ili će srušiti sve što je već konfigurisano" ili "da li main.tf ispunjava sve navedene zahtjeve zadatka" s obzirom da sam ponekad preskakao linije ili bio jednostavno nesiguran u ispravnost.
- Za init.sql nisam bio siguran da li PostgreSQL podržava iste if/else konstrukcije kao standardni SQL, pa sam koristio Claude AI kao debugger za potvrdu da li bi moj kod radio ispravno u PostgreSQL okruženju i ispravljao greške na osnovu dobivenih objašnjenja.
- Dijelove teksta koje nisam bio siguran kako pravilno formatirati ili gdje sam imao gramatičke greške slao sam Claude AI uz zahtjev da ih formatira, ispravi i vrati u bosanskom jeziku.

Zaključak

AI alati su korišteni isključivo kao podrška za učenje, generisanje početnih struktura i ubrzanje razvoja, dok je finalna implementacija prilagođena i integrisana ručno u skladu sa zahtjevima projekta.